BLOG CAKEPHP

Documentation Projet Complet

Camelin Mathéo — BTS SIO 2 — Option SLAM

1.1 Création du projet avec Composer.... 3
Étape 1 — Ouvrir le terminal dans le répertoire www.... 3
Étape 2 — Exécuter la commande Composer.... 3
Étape 3 — Importation dans NetBeans.... 3
1.2 Configuration de la base de données.... 3
Accéder à MySQL et créer l'utilisateur.... 3
Configurer config/app_local.php.... 3
1.3 Migrations Articles.... 4
Création initiale → Modification → Insertion de données.... 4
1.4 Modèle ArticlesTable.php.... 4
1.5 Contrôleur ArticlesController.php.... 4
1.6 Vues Articles.... 6
templates/Articles/index.php.... 6
templates/Articles/add.php.... 7
2.1 Migration de la table users.... 8
2.2 Modèle UsersTable.php.... 8
2.3 Hachage des mots de passe.... 8
2.4 Affichage dynamique avec jQuery.... 9
3.1 Migration de la table comments.... 10
3.2 Modèle CommentsTable.php.... 10
3.3 Intégration dans ArticlesController::detail().... 10
3.4 Suppression — CommentsController.php.... 11
4.1 Configuration — Application.php.... 12
4.2 AppController — accès non authentifié.... 12
4.3 Login / Logout — UsersController.php.... 13
Action login.... 13
Action logout.... 13
Vue login — templates/Users/login.php.... 13
4.4 Attribution automatique de l'utilisateur connecté.... 14
5.1 Migrations.... 15
Table tags.... 15
Table de jointure articles_tags.... 15
5.2 Modèles.... 15
TagsTable.php.... 15
ArticlesTagsTable.php.... 16
5.3 TagsController — CRUD complet.... 16
5.4 Vues Tags — templates/Tags/.... 17
index.php.... 18
add.php.... 18
edit.php.... 18
view.php.... 18
5.5 Association Tags ↔ Articles.... 19
ArticlesController::add() — récupérer et sauvegarder les tags.... 19
5.6 Affichage du nombre de tags / articles.... 19

PARTIE 1 — Création du projet et gestion des articles

1.1 Création du projet avec Composer

Étape 1 — Ouvrir le terminal dans le répertoire www

Aller dans **C:\wamp64\www** puis dans la barre de l'explorateur Windows, taper **cmd**.

Étape 2 — Exécuter la commande Composer

```
composer create-project --prefer-dist cakephp/app:5.* blog_app
```

Cette commande crée un nouveau projet CakePHP 5 dans le dossier blog_app.

Étape 3 — Importation dans NetBeans

- Ouvrir NetBeans
- File → New Project → PHP → PHP Application with Existing Sources
- Sources Folder : C:\wamp64\www\blog_app
- Project URL : http://localhost/blog_app/ | Index File : index.php

1.2 Configuration de la base de données

Accéder à MySQL et créer l'utilisateur

```
cd C:\wamp64\bin\mysql\mysql9.1.0\bin
mysql -u root

CREATE USER 'blog_app' IDENTIFIED BY 'M@theo97k1!';
CREATE DATABASE blog_app_bdd;
GRANT ALL PRIVILEGES ON blog_app_bdd.* TO blog_app;
```

Configurer config/app_local.php

```
'Datasources' => [
    'default' => [
        'host'          => 'localhost',
        'username'      => 'blog_app',
        'password'      => 'M@theo97k1!',
        'database'      => 'blog_app_bdd',
        'encoding'      => 'utf8',
        'timezone'      => 'UTC',
        'cacheMetadata' => true,
    ],
],
```

1.3 Migrations Articles

Création initiale → Modification → Insertion de données

```
cake migrations create InitialMigration   # crée la table articles
cake migrations migrate
cake migrations create AlterArticles       # ajoute les colonnes
cake migrations migrate
cake migrations create InsertArticles      # insère des données de test
cake migrations migrate
```

> ℹ️ Dans la méthode change() du fichier InsertArticles, utiliser $this->table('articles')->insert($rows)->saveData().

1.4 Modèle ArticlesTable.php

Créer : **src/Model/Table/ArticlesTable.php**

```
class ArticlesTable extends Table
{
    public function initialize(array $config): void
    {
        parent::initialize($config);
        $this->addBehavior('Timestamp');
        $this->hasMany('Comments');
    }
    public function validationDefault(Validator $validator): Validator
    {
        $validator
            ->notEmptyString('title',   'Veuillez renseigner un titre')
            ->notEmptyString('content', 'Veuillez renseigner une description');
        return $validator;
    }
}
```

1.5 Contrôleur ArticlesController.php

Créer : **src/Controller/ArticlesController.php**

```
class ArticlesController extends AppController
{
    public function initialize(): void
    {
```

```
        parent::initialize();
        $this->loadComponent('Flash');
    }

    public function index()
    {
        $mesArticles = $this->Articles->find()->all();
        $this->set(compact('mesArticles'));
    }

    public function detail($id = null)
    {
        try {
            $leArticle = $this->Articles->get($id);
            $this->set(compact('leArticle'));
        } catch (\Exception $e) {
            $this->Flash->error(__("L'article {0} n'existe pas", $id));
            return $this->redirect(['action' => 'index']);
        }
    }

    public function add()
    {
        $leNewArticle = $this->Articles->newEmptyEntity();
        if ($this->request->is('post')) {
            $leNewArticle = $this->Articles->patchEntity($leNewArticle,
$this->request->getData());
            if ($this->Articles->save($leNewArticle)) {
                $this->Flash->success(__('Article sauvegarde.'));
                return $this->redirect(['action' => 'index']);
            }
            $this->Flash->error(__('Impossible de sauvegarder.'));
        }
        $this->set(compact('leNewArticle'));
    }

    public function edit($id = null)
    {
        $leArticle = $this->Articles->get($id);
        if ($this->request->is(['post', 'put'])) {
```

```
            $this->Articles->patchEntity($leArticle,
$this->request->getData());
            if ($this->Articles->save($leArticle)) {
                $this->Flash->success(__('Article mis a jour.'));
                return $this->redirect(['action' => 'index']);
            }
        }
        $this->set(compact('leArticle'));
    }

    public function delete($id = null)
    {
        $this->request->allowMethod(['post', 'delete']);
        $leArticle = $this->Articles->get($id);
        $this->Articles->delete($leArticle);
        return $this->redirect(['action' => 'index']);
    }
}
```

1.6 Vues Articles

templates/Articles/index.php

```
<h1>Tous les articles du Blog</h1>
<?= $this->Html->link('Creer un article', ['action' => 'add'], ['class' =>
'button']) ?>
<table>
    <tr><th>Id</th><th>Titre</th><th>Date</th><th>Nb
Tags</th><th>Actions</th></tr>
    <?php foreach ($mesArticles as $article): ?>
    <tr>
        <td><?= $article->id ?></td>
        <td><?= $this->Html->link($article->title,
            ['action' => 'detail', $article->id]) ?></td>
        <td><?= $article->created->format(DATE_RFC850) ?></td>
        <td><?= count($article->tags) ?></td>
        <td>
            <?= $this->Html->link('Edit', ['action' => 'edit', $article->id])
?>
            <?= $this->Form->postLink('Supprimer', ['action' => 'delete',
$article->id],
                ['confirm' => 'Confirmer ?']) ?>
        </td>
```

```
    </tr>
    <?php endforeach; ?>
</table>
```

templates/Articles/add.php

```
<h1>Ajouter un article</h1>
<?php
    echo $this->Form->create($leNewArticle);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button('Sauvegarder l\'article');
    echo $this->Form->end();
?>
<?= $this->Html->link('Retour', ['action' => 'index'], ['class' => 'button'])
?>
```

PARTIE 2 — Gestion des utilisateurs

2.1 Migration de la table users

```
bin\cake bake migration CreateUsers name:string lastname:string
    age:integer email:string username:string password:string created modified
cake migrations migrate
cake migrations create InsertUsers
cake migrations migrate
```

2.2 Modèle UsersTable.php

```
class UsersTable extends Table
{
    public function initialize(array $config): void
    {
        $this->addBehavior('Timestamp');
        $this->hasMany('Articles');
        $this->hasMany('Comments');
    }
    public function validationDefault(Validator $validator): Validator
    {
        $validator
            ->notEmptyString('password', 'Mot de passe requis')
            ->notEmptyString('name',     'Nom requis')
            ->notEmptyString('lastname', 'Prenom requis')
            ->notEmptyString('email',    'Email requis')
            ->add('email', ['unique' => ['rule' => 'validateUnique',
                'provider' => 'table', 'message' => 'Mail deja pris']])
            ->notEmptyString('username', 'Username requis')
            ->add('username', ['unique' => ['rule' => 'validateUnique',
                'provider' => 'table', 'message' => 'Username deja pris']]);
        return $validator;
    }
}
```

2.3 Hachage des mots de passe

Les mots de passe ne doivent pas être stockés en clair. Installer le plugin d'authentification :

```
composer require "cakephp/authentication:^3.3"
composer update
```

Dans **src/Model/Entity/User.php**, ajouter le setter de mot de passe :

```
protected function _setPassword(string $password): ?string
{
    return (new DefaultPasswordHasher())->hash($password);
}
```

> ℹ️ Si des erreurs de hachage apparaissent, relancer composer update. La commande cake bake model Users permet de régénérer le modèle.

2.4 Affichage dynamique avec jQuery

Télécharger jQuery et le placer dans **webroot/js/jquery371min.js**.
Dans **templates/Articles/detail.php**, inclure le script :

```
<?php echo $this->Html->script('jquery371min'); ?>
<script>
$(document).ready(function () {
    $('#showcom').click(function () {
        if ($('#display').is(':visible') == false) { $('#display').show(); }
        else { $('#display').hide(); }
    });
});
</script>
```

> ℹ️ Pour les utilisateurs, placer une autour du tableau suffit pour le toggle jQuery, sans fichier element séparé.

PARTIE 3 — Gestion des commentaires

3.1 Migration de la table comments

```
cake migrations create CreateComments
# Dans le fichier :
$table = $this->table('comments');
$table->addColumn('title',      'string',  ['limit' => 255, 'null' => false])
      ->addColumn('content',    'text',    ['null' => false])
      ->addColumn('article_id', 'integer', ['null' => false])
      ->addForeignKey('article_id', 'articles', 'id', ['delete' => 'CASCADE'])
      ->create();
cake migrations migrate
```

3.2 Modèle CommentsTable.php

```
class CommentsTable extends Table
{
    public function initialize(array $config): void
    {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Articles');
        $this->belongsTo('Users');
    }
    public function validationDefault(Validator $validator): Validator
    {
        $validator
            ->notEmptyString('title',      'Titre requis')
            ->notEmptyString('content',    'Contenu requis')
            ->notEmptyString('article_id', 'article_id requis');
        return $validator;
    }
}
```

Dans **ArticlesTable.php**, ajouter la relation :

```
$this->hasMany('Comments');
```

3.3 Intégration dans ArticlesController::detail()

```
public function detail($id = null)
```

```
{
    $comments     = $this->fetchTable('Comments');
    $leNewComment = $comments->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewComment             = $comments->patchEntity($leNewComment,
$this->request->getData());
        $leNewComment->article_id = $id;
        if ($comments->save($leNewComment)) {
            $this->Flash->success(__('Commentaire sauvegarde.'));
            return $this->redirect(['action' => 'detail', $id]);
        }
        $this->Flash->error(__('Impossible d\'ajouter le commentaire.'));
    }
    try {
        $leArticle = $this->Articles->get($id, contain: [
            'Comments' => fn($q) => $q->order(['Comments.created asc'])
        ]);
    } catch (\Exception $ex) {
        $this->Flash->error(__("L'article {0} n'existe pas.", $id));
        return $this->redirect(['action' => 'index']);
    }
    $this->set(compact('leArticle', 'leNewComment'));
}
```

3.4 Suppression — CommentsController.php

```
public function delete($id = null)
{
    $this->request->allowMethod(['post', 'delete']);
    $leComm = $this->Comments->get($id);
    if ($this->Comments->delete($leComm)) {
        $this->Flash->success(__('Commentaire supprime.'));
    }
    return $this->redirect(['controller' => 'articles', 'action' => 'detail',
$leComm->article_id]);
}
```

- L'auteur du commentaire peut le supprimer
- L'auteur de l'article peut supprimer tous les commentaires de son article

PARTIE 4 — Système d'authentification

> ℹ️ Section ajoutée — basée sur le plugin cakephp/authentication:^3.3 installé en Partie 2.

4.1 Configuration — Application.php

```
class Application extends BaseApplication
    implements AuthenticationServiceProviderInterface
{
    public function bootstrap(): void
    {
        parent::bootstrap();
        $this->addPlugin('Authentication');
    }

    public function getAuthenticationService(ServerRequestInterface $request)
                    : AuthenticationServiceInterface
    {
        $service = new AuthenticationService();
        $service->setConfig(['unauthenticatedRedirect' => '/users/login']);
        $fields = [
            AbstractIdentifier::CREDENTIAL_USERNAME => 'username',
            AbstractIdentifier::CREDENTIAL_PASSWORD => 'password',
        ];
        $service->loadAuthenticator('Authentication.Session');
        $service->loadAuthenticator('Authentication.Form', [
            'fields' => $fields, 'loginUrl' => '/users/login',
        ]);
        $service->loadIdentifier('Authentication.Password', compact('fields'));
        return $service;
    }
}
```

4.2 AppController — accès non authentifié

```
public function beforeFilter(\Cake\Event\EventInterface $event)
{
    parent::beforeFilter($event);
```

```
    $this->Authentication->allowUnauthenticated(['view', 'index', 'detail']);
}
```

4.3 Login / Logout — UsersController.php

Action login

```
public function beforeFilter(\Cake\Event\EventInterface $event)
{
    parent::beforeFilter($event);
    $this->Authentication->allowUnauthenticated(['login', 'add']);
}

public function login()
{
    $result = $this->Authentication->getResult();
    if ($result && $result->isValid()) {
        $target = $this->Authentication->getLoginRedirect() ?? '/';
        return $this->redirect($target);
    }
    if ($this->request->is('post') && !$result->isValid()) {
        $this->Flash->error('Invalid username or password');
    }
}
```

Action logout

```
public function logout()
{
    $this->Authentication->logout();
    return $this->redirect(['controller' => 'Users', 'action' => 'login']);
}
```

Vue login — templates/Users/login.php

```
<div class='users form content'>
    <?= $this->Form->create() ?>
    <fieldset>
        <legend>Connexion</legend>
        <?= $this->Form->control('username') ?>
        <?= $this->Form->control('password') ?>
    </fieldset>
    <?= $this->Form->button('Login') ?>
    <?= $this->Form->end() ?>
```

```
</div>
```

4.4 Attribution automatique de l'utilisateur connecté

```
// Dans ArticlesController::add()
$leNewArticle->user_id = $this->request->getAttribute('identity')->id;
```

ℹ️ Dans la vue add.php, commenter la ligne de sélection manuelle du user_id.

PARTIE 5 — Système de Tags

5.1 Migrations

Table tags

```
cake migrations create CreateTags
$table = $this->table('tags');
$table->addColumn('title',    'string',   ['limit' => 255, 'null' => false])
      ->addColumn('created',  'datetime', ['null' => false])
      ->addColumn('modified', 'datetime', ['null' => false])
      ->addIndex('title', ['unique' => true])
      ->create();
cake migrations migrate
```

Table de jointure articles_tags

```
cake migrations create CreateArticlesTags
$table = $this->table('articles_tags');
$table->addColumn('article_id', 'integer', ['null' => false])
      ->addColumn('tag_id',     'integer', ['null' => false])
      ->addColumn('priority',   'integer', ['null' => false])
      ->addColumn('modified',   'string',  ['null' => false])
      ->addForeignKey('article_id', 'articles', 'id')
      ->addForeignKey('tag_id',     'tags',     'id')
      ->create();
cake migrations migrate
```

5.2 Modèles

TagsTable.php

```
class TagsTable extends Table
{
    public function initialize(array $config): void
    {
        $this->setTable('tags'); $this->setDisplayField('title');
        $this->addBehavior('Timestamp');
        $this->belongsToMany('Articles', [
            'foreignKey'       => 'tag_id',
            'targetForeignKey' => 'article_id',
            'joinTable'        => 'articles_tags',
```

```
        ]);
    }
    public function validationDefault(Validator $validator): Validator
    {
        $validator->scalar('title')->maxLength('title', 255)
            ->requirePresence('title', 'create')->notEmptyString('title')
            ->add('title', 'unique', ['rule' => 'validateUnique', 'provider' =>
'table']);
        return $validator;
    }
}
```

ArticlesTagsTable.php

```
class ArticlesTagsTable extends Table
{
    public function validationDefault(Validator $validator): Validator
    {
        $validator->integer('priority')->requirePresence('priority', 'create')
            ->notEmptyString('priority')
            ->add('priority', 'validValue', ['rule' => ['range', 1, 5]]);
        return $validator;
    }
}
```

5.3 TagsController — CRUD complet

```
class TagsController extends AppController
{
    // index() : liste des tags avec nombre d'articles
    public function index()
    {
        $mesTags = $this->Tags->find()->contain(['Articles'])->all();
        $this->set(compact('mesTags'));
    }

    // view($id) : affiche un tag et ses articles
    public function view($id = null)
    {
        $leTag = $this->Tags->get($id, contain: ['Articles']);
        $this->set(compact('leTag'));
    }
```

```
    // add() : créer un tag
    public function add()
    {
        $leNewTag = $this->Tags->newEmptyEntity();
        if ($this->request->is('post')) {
            $leNewTag = $this->Tags->patchEntity($leNewTag,
$this->request->getData());
            if ($this->Tags->save($leNewTag)) {
                $this->Flash->success(__('Tag cree.')); return
$this->redirect(['action' => 'index']);
            }
        }
        $this->set(compact('leNewTag'));
    }

    // edit($id) : modifier un tag
    public function edit($id = null)
    {
        $leTag = $this->Tags->get($id);
        if ($this->request->is(['post', 'put'])) {
            $this->Tags->patchEntity($leTag, $this->request->getData());
            if ($this->Tags->save($leTag)) {
                $this->Flash->success(__('Tag mis a jour.')); return
$this->redirect(['action' => 'index']);
            }
        }
        $this->set(compact('leTag'));
    }

    // delete($id) : supprimer un tag
    public function delete($id = null)
    {
        $this->request->allowMethod(['post', 'delete']);
        $leTag = $this->Tags->get($id);
        $this->Tags->delete($leTag);
        return $this->redirect(['action' => 'index']);
    }
}
```

5.4 Vues Tags — templates/Tags/

index.php

```
<h1>Liste des Tags</h1>
<?= $this->Html->link('Ajouter un tag', ['action' => 'add']) ?>
<table>
    <tr><th>Id</th><th>Titre</th><th>Nb
Articles</th><th>Modifie</th><th>Actions</th></tr>
    <?php foreach ($mesTags as $tag): ?>
    <tr>
        <td><?= $tag->id ?></td>
        <td><?= h($tag->title) ?></td>
        <td><?= count($tag->articles) ?></td>
        <td><?= $tag->modified->format(DATE_RFC850) ?></td>
        <td>
            <?= $this->Html->link('View', ['action' => 'view', $tag->id]) ?>
            <?= $this->Html->link('Edit', ['action' => 'edit', $tag->id]) ?>
            <?= $this->Form->postLink('Delete', ['action' => 'delete',
$tag->id],
                ['confirm' => 'Confirmer ?']) ?>
        </td>
    </tr>
    <?php endforeach; ?>
</table>
```

add.php

```
<h1>Ajouter un tag</h1>
<?php
    echo $this->Form->create($leNewTag);
    echo $this->Form->control('title', ['label' => 'Titre du tag']);
    echo $this->Form->button('Sauvegarder');
    echo $this->Form->end();
?>
```

edit.php

```
<h1>Modifier le tag '<?= h($leTag->title) ?>'</h1>
<?php
    echo $this->Form->create($leTag);
    echo $this->Form->control('title');
    echo $this->Form->button('Mettre a jour');
    echo $this->Form->end();
?>
```

view.php

```
<h1>Tag : <?= h($leTag->title) ?></h1>
```

```
<h3>Articles associes (<?= count($leTag->articles) ?>) :</h3>
<ul>
    <?php foreach ($leTag->articles as $article): ?>
    <li><?= $this->Html->link($article->title,
        ['controller' => 'Articles', 'action' => 'detail', $article->id])
?></li>
    <?php endforeach; ?>
</ul>
```

5.5 Association Tags ↔ Articles

ArticlesController::add() — récupérer et sauvegarder les tags

```
$mesTags = $this->fetchTable('Tags')
               ->find('list', keyField: 'id', valueField: 'title')
               ->toArray();


if ($this->request->is('post')) {
    $data    = $this->request->getData();
    $newData = ['title' => $data['title'], 'content' => $data['content'],
'tags' => []];
    if (!empty($data['tags']['_ids'])) {
        foreach ($data['tags']['_ids'] as $tagId) {
            foreach ($data['tags']['_joinData'] as $tag => $prio) {
                if ($tag == $tagId)
                    $newData['tags'][] = ['id' => $tagId, '_joinData' =>
$prio];
            }
        }
    }
    $leNewArticle = $this->Articles->patchEntity($leNewArticle, $newData);
    if ($this->Articles->save($leNewArticle)) { ... }
}
```

5.6 Affichage du nombre de tags / articles

Dans **index.php de Articles** — colonne Nb Tags :

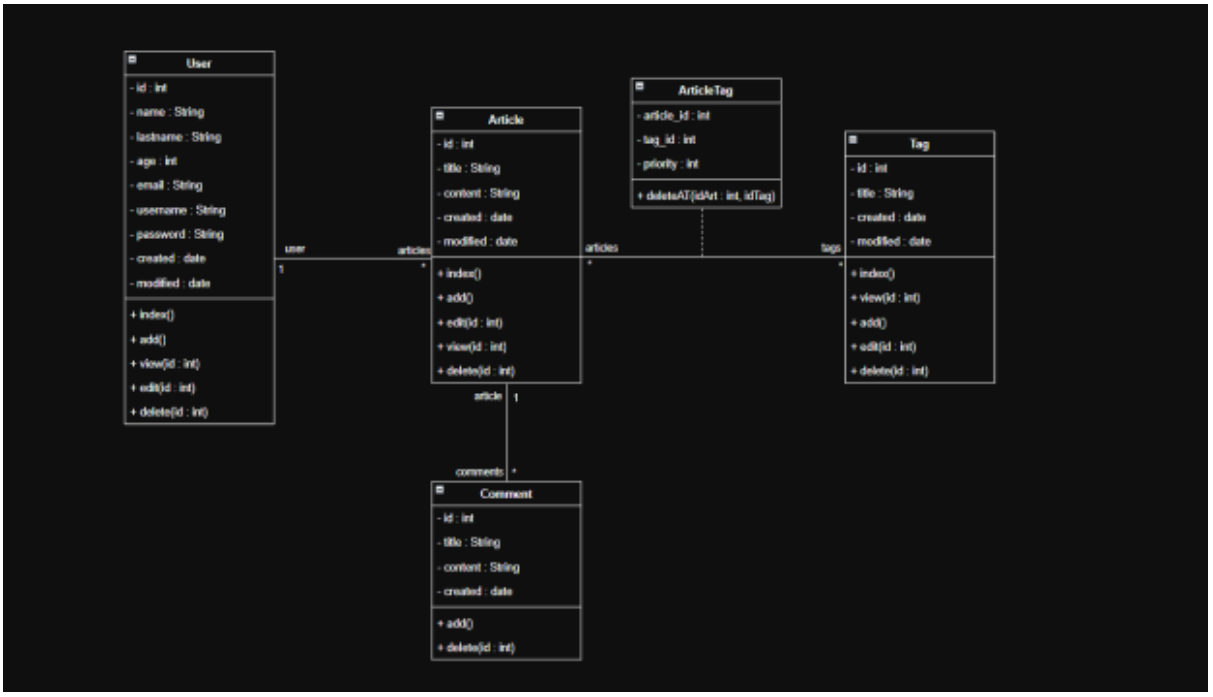
```
<td><?= count($article->tags) ?></td>
```

Dans **index.php de Tags** — colonne Nb Articles :

```
<td><?= count($tag->articles) ?></td>
```

> ℹ️ Pour que count() fonctionne, le contain(['Tags']) ou contain(['Articles']) doit être présent dans la requête du contrôleur.

DIAGRAMME DE CLASSES FINAL



RÉCAPITULATIF DES ÉTAPES DE BASE

1	**Créer un fichier de migration** cake migrations create Nom → modifier le fichier → cake migrations migrate (rollback pour annuler)
2	**Créer le modèle (Table)** src/Model/Table/NomTable.php → relations (hasMany, belongsTo, belongsToMany) + validateurs
3	**Créer le contrôleur** src/Controller/NomController.php → actions index, add, edit, delete, view/detail
4	**Créer les vues** templates/Nom/ → un fichier PHP par action du contrôleur
5	**Tester** Ouvrir http://localhost/blog_app/ et vérifier chaque fonctionnalité